Раздел 1. Цель работы и задачи

Лабораторная работа № 10. Реализация механизма вывода и объяснения решений. Вариант 10: промышленное оборудование и предиктивное обслуживание.

Цель работы - реализовать прямой вывод по базе из 24 правил работы 9, связать его с PostgreSQL, получить объяснения и оценить качество по фактическим меткам AI4I 2020.

1. обосновать тип вывода и реализовать сопоставление, выбор и применение правил;
2. реализовать детерминированное разрешение конфликтов и остановку;
3. загрузить наблюдения из хранилища, исключив метки из входных фактов;
4. получить 3–5 объяснений, проверить не менее 50 примеров и разобрать 2–3 ошибки.

Раздел 2. Выбор типа вывода

Таблица 1 - Критерии выбора

Критерий	Предметная область	Решение
Направление решения	Показания датчиков → симптом → риск → обслуживание	Прямой вывод
Есть ли заданная цель?	Нужно получить все симптомы и выбрать действие	Нет единственной цели для обратного поиска
Данные поступают постепенно?	В этой работе обрабатывается готовое наблюдение; в эксплуатации — поток	Новый запуск на каждое наблюдение
Тип знаний	Пороговые условия из справочников и экспертные рекомендации	Декларативные правила без нейронных сетей

Использован стратифицированный прямой вывод по трём стадиям. Сначала завершается диагностика, затем определяется уровень риска, после этого — рекомендация. Правила не читают выводы своей или более поздней стадии. Это предотвращает применение отрицательного условия «симптома нет» до завершения его вычисления.



Рисунок 1 - Схема механизма вывода

Раздел 3. Рабочая память и алгоритм

Функция record_to_facts формирует разрешённый набор из 19 полей, проверяет типы и диапазоны, отбрасывает поля целевых меток. Исходный словарь не изменяется. Выведенные флаги только добавляются; уже записанный факт не перезаписывается менее приоритетным.

Каждое правило выбирается не более одного раза. Если правило не добавило фактов из-за конфликта, поиск продолжается: остановка в таком случае потеряла бы другие применимые правила. Неподвижная точка достигнута, когда ни в одной стадии не остаётся нерассмотренных применимых правил. При превышении max_steps возникает ошибка; незавершённый результат не выдаётся как успешный.

Число применённых правил — число шагов, добавивших хотя бы один факт. Число выбранных правил включает повторные по смыслу и

заблокированные заключения. Обе величины сохранены, чтобы не смешивать активность базы с объёмом нового знания.

Ключевые функции

```
def match_rules(facts, rules, fired, stage):
    return [r for r in rules if r['stage']==stage and r['id'] not in fired and
matches(r,facts)]

def resolve_conflicts(applicable):
    """Больший приоритет, затем больше условий, затем лексикографически меньший
ID."""
    return min(applicable, key=lambda r: (-r['priority'], -len(r['conditions']),
r['id']))

def apply_rule(rule, facts):
    """Первое заключение о поле сохраняется, несовместимые записываются в
трассу."""
    added, blocked = {}, {}
    for key,value in rule['then'].items():
        if key not in facts:
            facts[key] = value
            added[key] = value
        elif facts[key] != value:
            blocked[key] = {'retained': facts[key], 'rejected': value}
    return added, blocked

def infer(record, rules=None, max_steps=100):
    rules = load_rules() if rules is None else rules
    facts = record_to_facts(record)  # Разрешённый список, целевые метки отброшены.
    original = deepcopy(facts)
    fired, trace = set(), []
    for stage in (1,2,3):
        while True:
            applicable = match_rules(facts,rules,fired,stage)
            if not applicable:
                break
            if len(trace) >= max_steps:
                raise RuntimeError('Превышен лимит вывода; результат не завершён')
            selected = resolve_conflicts(applicable)
            premises = []
            for c in selected['conditions']:
                premises.append({'condition': condition_text(c),
                                 'observed': facts.get(c['field'],
'<отсутствует>'),
                                 'threshold': facts[c['ref']] if 'ref' in c else
c.get('value')})
            new, blocked = apply_rule(selected,facts)
            fired.add(selected['id'])
            trace.append(dict(rule=selected['id'],stage=stage,
                              conflict_set=[r['id'] for r in applicable],

explanation=selected['explanation'],premises=premises,
                              new_facts=new,blocked=blocked))
            # Даже если правило ничего не добавило, оно помечено сработавшим.
            # Проверяем оставшиеся; иначе конфликт мог бы остановить полезный
вывод.
    assert all(facts[k]==v for k,v in original.items())
    facts.setdefault('predicted_failure',False)
    if 'risk_level' not in facts or 'action' not in facts:
        raise RuntimeError('База не покрывает ситуацию')
    return {'facts':facts,'trace':trace,
            'applied_count':sum(bool(t['new_facts']) for t in trace),
            'terminated':'fixed_point'}
```

Раздел 4. Стратегия разрешения конфликтов

Таблица 2 - Выбор правила

Параметр	Реализация
Основная стратегия	Наибольший priority
Равный приоритет	Большее число условий; затем меньший строковый ID
Конфликт значений одного поля	Сохраняется ранее выведенное приоритетное значение; отклонение попадает в trace.blocked
Несколько симптомов	Сохраняются все флаги, но выбирается одно действие
Обоснование	Срочная диагностика PWF/OSF важнее профилактических предупреждений

Например, одновременные PWF и OSF не взаимно исключаются. На стадии 1 будут выведены оба флага, на стадии 2 — critical, на стадии 3 — диагностика привода (приоритет 100 против 95). Более низкие рекомендации отражаются в объяснении как отклонённые, а не скрываются.

Раздел 5. Интеграция с PostgreSQL и протокол оценки

Общий модуль tools/pdm_data.py выполняет SELECT с JOIN fact_observation, dim_quality_class и dim_equipment, упорядоченный по udi. Получены все 10 000 строк. DB_HOST, DB_PORT, DB_NAME, DB_USER и DB_PASSWORD берутся из окружения. Значение пароля не сохраняется в отчётах или результатах.

```
SELECT f.*, q.class_code, q.osf_limit,
       e.power_min_w, e.power_max_w, e.hdf_temp_diff_min_k,
       e.hdf_speed_max_rpm, e.twf_wear_min_min, e.twf_wear_max_min
FROM dwh.fact_observation f
JOIN dwh.dim_quality_class q USING (class_key)
JOIN dwh.dim_equipment e USING (equipment_key)
ORDER BY f.udi;
```

Пороговые правила зафиксированы в работе 9 до оценки. Обучения и подбора порогов на метках здесь нет. Измеряется воспроизведение известных закономерностей синтетического AI4I, а не качество прогноза будущих отказов на независимом станке. Метки используются только после вывода при вычислении TP/TN/FP/FN и выборе примеров ошибок.

Основной бинарный прогноз — наличие условия PWF, OSF или HDF. Зона TWF влияет на обслуживание и средний риск, но не превращается в

положительный диагноз. Отдельно вычислен вариант с включением зоны TWF в бинарный прогноз.

```
source_sha256 = 6d55b6f942595bed7ebe0c65e0c11cdc6b3fb35810715550072da81b9bf2e2b8
rules_sha256  = c8a45e08219634ee684d38d51e4d811bdf1deb2828aae89b8a01016f3d77d356
```

Раздел 6. Объяснения решений: пять примеров

Трасса хранит конфликтное множество, выбранное правило, фактические значения проверенных признаков, добавленные факты и отклонённые заключения. Функция explain формирует HOW-объяснение уже полученного результата. Интерактивный WHY-опрос не реализован, поскольку все признаки загружаются из БД.

```
def explain(result):
    facts = result['facts']
    lines = [f"UDI {int(facts['udi'])}: риск={facts['risk_level']};
действие={facts['action']}; "
             f"прогноз отказа={facts['predicted_failure']}."]
    for t in result['trace']:
        if not t['new_facts'] and not t['blocked']:
            continue
        conditions = '; '.join(f"{p['condition']} (факт: {p['observed']})" for p in
t['premises'])
        lines.append(f"{t['rule']}: {conditions}. {t['explanation']} Новые факты:
{t['new_facts']}.")
        if t['blocked']:
            lines.append(f"  Сохранено более приоритетное заключение: {t['blocked']}.")
    if not facts['predicted_failure']:
        lines.append('Условия PWF/OSF/HDF не доказаны: прогноз 0 по замкнутому миру; '
                     'это не исключает TWF и неуказанный тип отказа.')
    return '\n'.join(lines)
```

6.1. PWF, наблюдение UDI 51

Решение: Диагностика привода.

```
UDI 51: риск=critical; действие=drive_diagnostics; прогноз отказа=True.
R001: power_w < power_min_w (факт: 1378.2). Мощность ниже допустимой: условие PWF. Новые
факты: {'pwf_risk': True}.
R009: pwf_risk = True (факт: True). Признак pwf_risk: уровень риска critical. Новые факты:
{'risk_level': 'critical', 'predicted_failure': True}.
R017: pwf_risk = True (факт: True). Диагностика привода Новые факты: {'action':
'drive_diagnostics'}.
```

6.2. OSF, наблюдение UDI 161

Решение: Снижение подачи, внеочередной осмотр.

```
UDI 161: риск=critical; действие=reduce_load; прогноз отказа=True.
R003: wear_torque > osf_limit (факт: 13111.2). Износ × момент превышает порог класса: условие
OSF. Новые факты: {'osf_risk': True}.
R005: tool_wear_min ≥ twf_wear_min_min (факт: 216.0); tool_wear_min ≤ twf_wear_max_min (факт:
216.0). Инструмент в зоне риска TWF; это не доказательство отказа. Новые факты: {'twf_risk':
True}.
R008: tool_wear_min ≥ 180 (факт: 216.0). Высокий накопленный износ: подготовить инструмент.
Новые факты: {'worn_tool': True}.
R010: osf_risk = True (факт: True). Признак osf_risk: уровень риска critical. Новые факты:
{'risk_level': 'critical', 'predicted_failure': True}.
R012: twf_risk = True (факт: True). Признак twf_risk: уровень риска medium. Новые факты: {}.
  Сохранено более приоритетное заключение: {'risk_level': {'retained': 'critical', 'rejected':
'medium'}}.
R015: worn_tool = True (факт: True). Признак worn_tool: уровень риска low. Новые факты: {}.
```

```
  Сохранено более приоритетное заключение: {'risk_level': {'retained': 'critical', 'rejected':
'low'}}.
R018: osf_risk = True (факт: True). Снижение подачи, внеочередной осмотр Новые факты:
{'action': 'reduce_load'}.
R020: twf_risk = True (факт: True). Плановая замена инструмента Новые факты: {}.
  Сохранено более приоритетное заключение: {'action': {'retained': 'reduce_load', 'rejected':
'plan_tool_replacement'}}.
R023: worn_tool = True (факт: True). Подготовить сменный инструмент Новые факты: {}.
  Сохранено более приоритетное заключение: {'action': {'retained': 'reduce_load', 'rejected':
'prepare_tool'}}.
```

6.3. HDF, наблюдение UDI 3788

Решение: Проверка системы охлаждения.

```
UDI 3788: риск=high; действие=check_cooling; прогноз отказа=True.
R004: temp_diff_k < hdf_temp_diff_min_k (факт: 8.5); rot_speed_rpm < hdf_speed_max_rpm (факт:
1377.0). Малый теплоотвод при низких оборотах: условие HDF. Новые факты: {'hdf_risk': True}.
R011: hdf_risk = True (факт: True). Признак hdf_risk: уровень риска high. Новые факты:
{'risk_level': 'high', 'predicted_failure': True}.
R019: hdf_risk = True (факт: True). Проверка системы охлаждения Новые факты: {'action':
'check_cooling'}.
```

6.4. Риск TWF без отказа, наблюдение UDI 74

Решение: Плановая замена инструмента.

```
UDI 74: риск=medium; действие=plan_tool_replacement; прогноз отказа=False.
R005: tool_wear_min ≥ twf_wear_min_min (факт: 200.0); tool_wear_min ≤ twf_wear_max_min (факт:
200.0). Инструмент в зоне риска TWF; это не доказательство отказа. Новые факты: {'twf_risk':
True}.
R008: tool_wear_min ≥ 180 (факт: 200.0). Высокий накопленный износ: подготовить инструмент.
Новые факты: {'worn_tool': True}.
R012: twf_risk = True (факт: True). Признак twf_risk: уровень риска medium. Новые факты:
{'risk_level': 'medium'}.
R015: worn_tool = True (факт: True). Признак worn_tool: уровень риска low. Новые факты: {}.
  Сохранено более приоритетное заключение: {'risk_level': {'retained': 'medium', 'rejected':
'low'}}.
R020: twf_risk = True (факт: True). Плановая замена инструмента Новые факты: {'action':
'plan_tool_replacement'}.
R023: worn_tool = True (факт: True). Подготовить сменный инструмент Новые факты: {}.
  Сохранено более приоритетное заключение: {'action': {'retained': 'plan_tool_replacement',
'rejected': 'prepare_tool'}}.
Условия PWF/OSF/HDF не доказаны: прогноз 0 по замкнутому миру; это не исключает TWF и
неуказанный тип отказа.
```

6.5. Нет известных признаков, наблюдение UDI 1

Решение: Продолжить наблюдение.

```
UDI 1: риск=low; действие=continue_monitoring; прогноз отказа=False.
R016: нет pwf_risk (факт: <отсутствует>); нет osf_risk (факт: <отсутствует>); нет hdf_risk
(факт: <отсутствует>); нет twf_risk (факт: <отсутствует>); нет near_power (факт:
<отсутствует>); нет near_osf (факт: <отсутствует>); нет worn_tool (факт: <отсутствует>).
Известные условия риска отсутствуют. Новые факты: {'risk_level': 'low'}.
R024: risk_level = low (факт: low). Продолжить наблюдение; случайный и неизвестный отказ не
исключены. Новые факты: {'action': 'continue_monitoring'}.
Условия PWF/OSF/HDF не доказаны: прогноз 0 по замкнутому миру; это не исключает TWF и
неуказанный тип отказа.
```

Раздел 7. Тестирование и метрики

test_inference.py проверяет 18 независимо заданных сценариев, все 24 правила имеют хотя бы один пример, добавляющий их заключение. Проверены границы порогов, совместные симптомы, приоритет,

специфичность и ID, неизменность входа, перестановка базы, отсутствие утечки метки, ошибки ввода, дубликаты и лимит шагов.

Таблица 3 - Результаты на полном хранилище

Показатель	Значение
Число наблюдений	10000
Верно классифицированы	9936
Положительные метки	339
TP / TN / FP / FN	275 / 9661 / 0 / 64
Accuracy, %	99,36
Precision, %	100,00
Recall, %	81,12
F1, %	89,58
Среднее число применённых правил	2,32
Среднее число выбранных правил	2,62
Версия Python	3.12.14

Accuracy = (TP + TN) / N; precision = TP / (TP + FP); recall = TP / (TP + FN); F1 = 2 · precision · recall / (precision + recall). Положительный класс — отказ. При нулевом знаменателе метрика принята равной нулю, в частности для константного отрицательного прогноза.

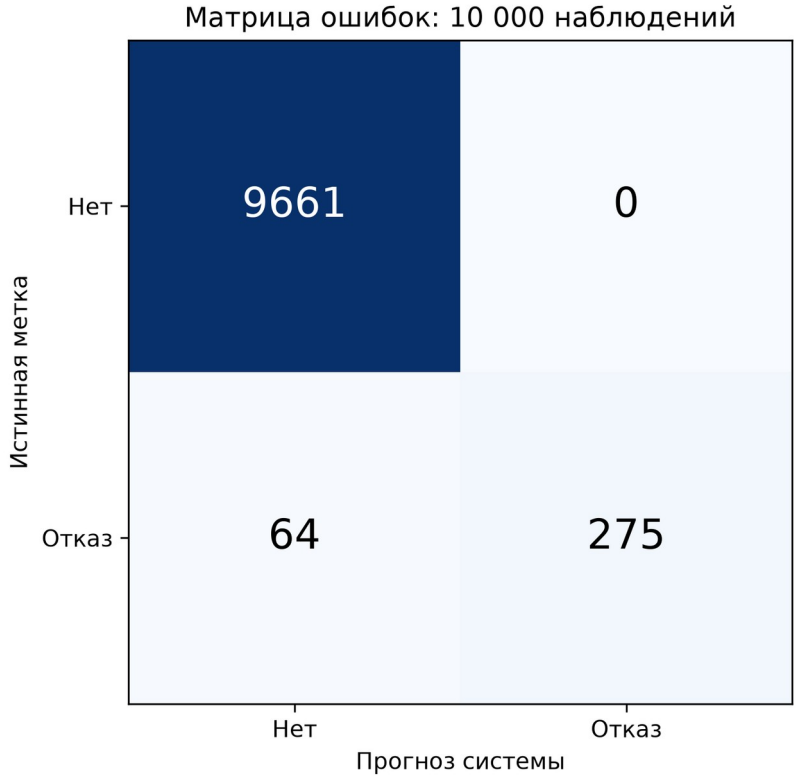


Рисунок 2 - Матрица ошибок бинарного прогноза

Таблица 4 - Сравнение политик бинарного прогноза

Политика	Accuracy, %	Precision, %	Recall, %	F1, %
PWF / OSF / HDF	99,36	100,00	81,12	89,58
Также зона TWF	93,10	32,29	94,40	48,12
Всегда нет отказа	96,61	0,00	0,00	0,00

Константный отрицательный прогноз имеет высокую accuracy из-за редкости отказов, но нулевую полноту. Включение TWF увеличивает полноту ценой большого числа ложных тревог. Поэтому оценка по одной accuracy и интерпретация износа как достоверного отказа были бы вводящими в заблуждение.

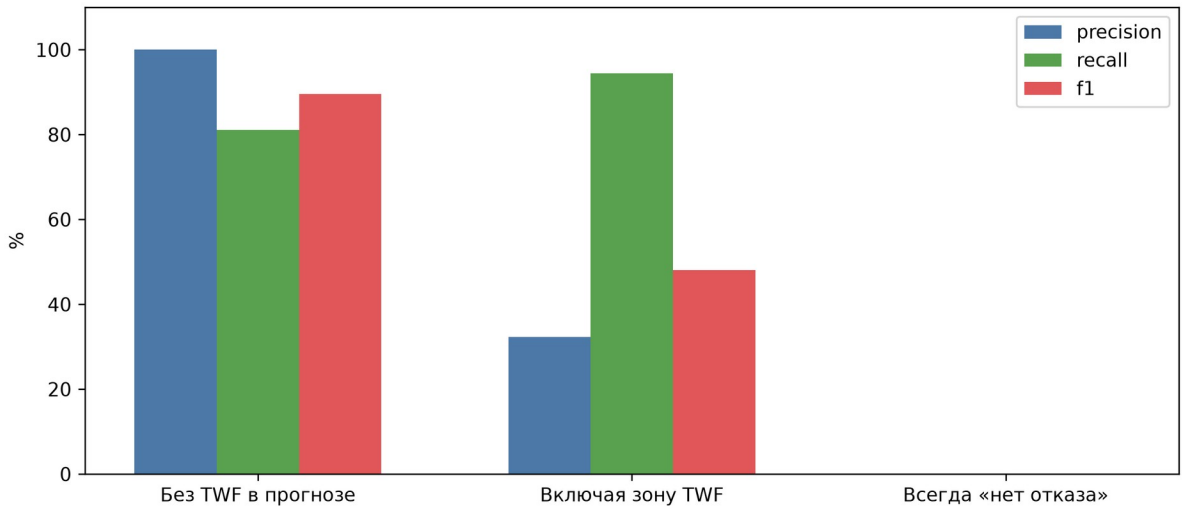


Рисунок 3 - Влияние зоны TWF на precision, recall и F1

Таблица 5 - Качество отдельных условий относительно меток подтипов

Условие	TP / FP / FN	Precision, %	Recall, %	F1, %
PWF	95 / 0 / 0	100,00	100,00	100,00
OSF	98 / 0 / 0	100,00	100,00	100,00
HDF	103 / 0 / 12	100,00	89,57	94,50
TWF	43 / 747 / 3	5,44	93,48	10,29

Раздел 8. Анализ трёх ошибок

8.1. Граница HDF, UDI 3237

Истинная метка machine_failure = 1, прогноз = 0, риск = medium. Действие: Проверить запас мощности.

Таблица 6 - Признаки ошибочного наблюдения

Признак	Значение
temp_diff_k	8,6
rot_speed_rpm	1 342,0
power_w	8 769,3
tool_wear_min	113,0
wear_torque	7 051,2
osf_limit	12 000,0

Причина: NUMERIC хранит 8,6 точно; правило < 8,6 не воспроизводит погрешность исходных float.

Возможное улучшение: Сохранить исходную точность в ETL либо отдельно согласовать <= и оценить ложные срабатывания.

8.2. Стохастический TWF, UDI 78

Истинная метка machine_failure = 1, прогноз = 0, риск = medium. Действие: Плановая замена инструмента.

Таблица 7 - Признаки ошибочного наблюдения

Признак	Значение
temp_diff_k	10,1
rot_speed_rpm	1 455,0
power_w	6 292,8
tool_wear_min	208,0
wear_torque	8 590,4
osf_limit	11 000,0

Причина: Зона износа — предупреждение; случайный факт отказа не определяется текущими показаниями.

Возможное улучшение: Добавить историю обслуживания/вибрации; оценивать калиброванный риск, а не объявлять всю зону отказом.

8.3. Неуказанный тип, UDI 1438

Истинная метка machine_failure = 1, прогноз = 0, риск = low. Действие: Продолжить наблюдение.

Таблица 8 - Признаки ошибочного наблюдения

Признак	Значение
temp_diff_k	11,1
rot_speed_rpm	1 439,0
power_w	6 811,3

Признак	Значение
tool_wear_min	40,0
wear_torque	1 808,0
osf_limit	13 000,0

Причина: Метка положительна без объясняющего подтипа; в разрешённых входах нет соответствующего симптома.

Возможное улучшение: Разобрать первичную разметку; получить дополнительные датчики и экспертные правила.

Изменения порогов не внесены после просмотра ошибок: это сохранило исходный протокол оценки. Любую новую политику следует заранее зафиксировать и проверить отдельно; отсутствующий в датчиках случайный исход нельзя восстановить добавлением правила по истинной метке.

Раздел 9. Выводы

Реализован завершающийся прямой вывод с трассировкой и приоритетным разрешением конфликтов. На 10 000 наблюдениях precision составила 100,00 %, recall — 81,12 %, F1 — 89,58 %. Пропущено 64 отказа; ложных положительных решений нет. Погрешность на границе HDF и стохастический TWF согласуются с результатами предыдущих работ. Система пригодна как учебный объяснимый прототип, но эти результаты не доказывают способность прогнозировать будущие отказы реального оборудования.

Раздел 10. Ответы на контрольные вопросы

1. Прямой вывод повторяет сопоставление, выбор и применение правила. Неподвижная точка — состояние, в котором новых фактов получить нельзя. Здесь каждому правилу разрешён один выбор, а стадии конечны.

2. Прямой вывод идёт от измерений к решениям; обратный — от цели к подцелям. Мониторинг естественнее реализовать прямым выводом, проверку одной гипотезы — обратным.
3. Трассировка фиксирует основания и порядок решений, позволяет объяснять результат и находить неверные знания или приоритеты.
4. F1 — гармоническое среднее precision и recall. При редких отказах она полезнее accuracy, которая может быть высокой у системы, не находящей ни одного отказа.
5. Низкое качество вызывают неполная база, ошибочные границы, конфликтующие правила, неточная разметка и недостаточные признаки. Исправления требуют экспертной проверки и повторной оценки без утечки меток.

Список использованных источников

1. Джарратано Дж., Райли Г. Экспертные системы: принципы разработки и программирование. 4-е изд. М.: Вильямс, 2007.
2. Гаврилова Т. А., Хорошевский В. Ф. Базы знаний интеллектуальных систем. СПб.: Питер, 2000.
3. AI4I 2020 Predictive Maintenance Dataset. UCI. DOI: 10.24432/C5HS5C.
4. Материалы сквозного проекта: работы 4–9; PostgreSQL, схема dwh.
5. Python documentation: operator. https://docs.python.org/3/library/operator.html (дата обращения: 09.11.2026).